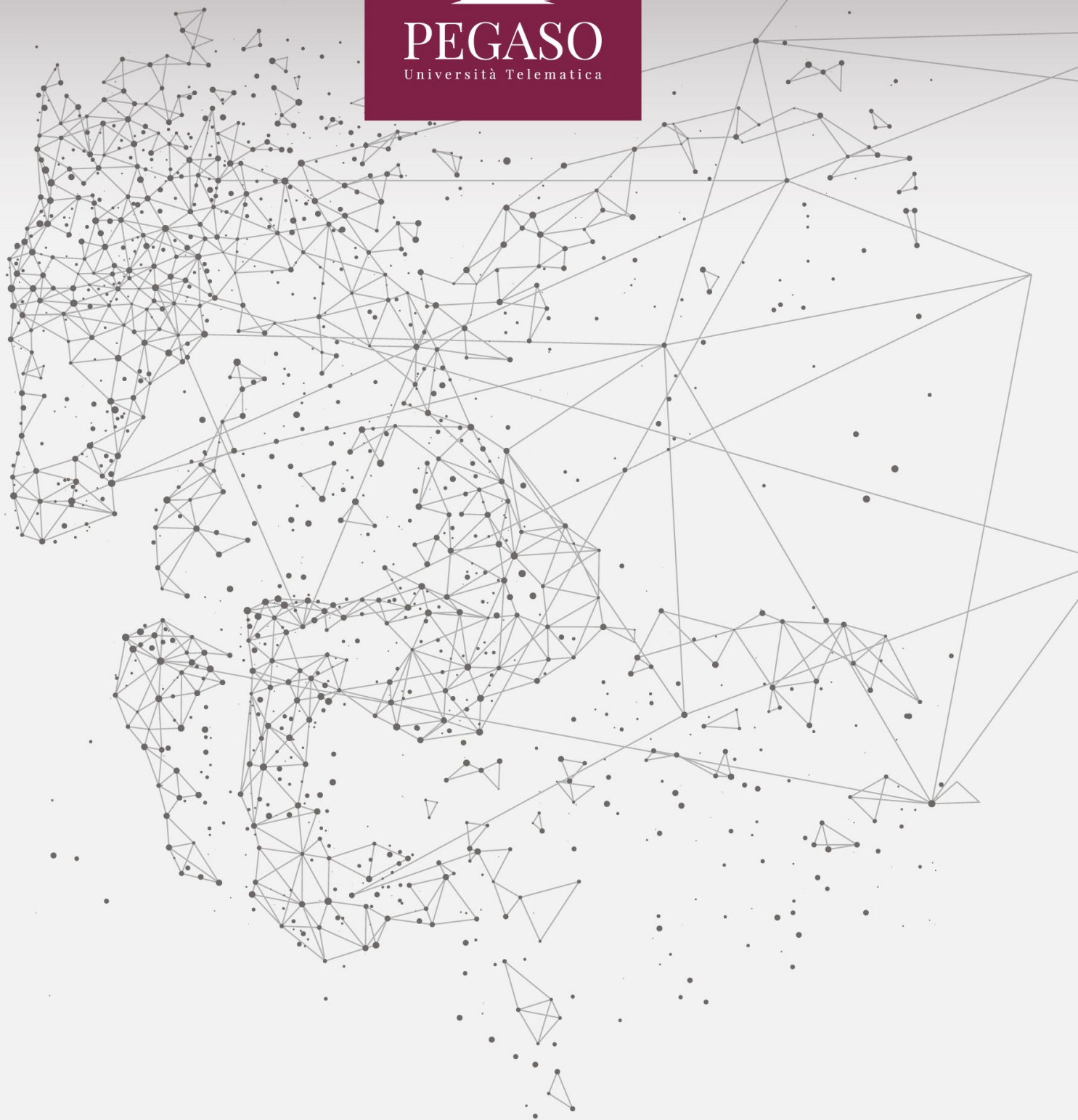




PEGASO
Università Telematica



Indice

1. IL CICLO DI VITA DEL SOFTWARE	3
2. LA PROGETTAZIONE DEL SOFTWARE.....	7
3. LA PROGRAMMAZIONE STRUTTURATA	11
BIBLIOGRAFIA	14

1. Il ciclo di vita del software

La progettazione dei programmi è un processo intellettuale che richiede creatività ed intuito, essenziali per sviluppare soluzioni efficienti e innovative a problemi complessi. Questa fase di progettazione è spesso più impegnativa rispetto alla successiva codifica dell'algoritmo con un linguaggio di programmazione, poiché implica la creazione e l'ottimizzazione di procedure logiche e sequenziali che guidano il calcolo o la risoluzione del problema. Nonostante l'importanza di strutturare algoritmi efficaci, non esiste un metodo predefinito o un algoritmo universale per la progettazione stessa degli algoritmi; essa richiede piuttosto un approccio personalizzato e adattabile a ciascun contesto specifico, sfruttando al meglio le capacità creative e analitiche del progettista.

Nella progettazione dei programmi è essenziale valutare possibili algoritmi diversi al fine di identificare la soluzione ottimale. La valutazione in genere si basa su tre principali criteri: la correttezza dell'algoritmo, che verifica se la soluzione prodotta rispetta pienamente le specifiche del problema; la complessità computazionale, che mira a determinare come l'algoritmo utilizzi efficientemente le risorse disponibili, come tempo e spazio; infine, la scalabilità, che considera come l'algoritmo si comporta con l'aumentare delle dimensioni del problema.

Questi criteri sono fondamentali per garantire che l'algoritmo scelto non solo produca risultati accurati ma sia anche efficiente e adatto alle necessità specifiche dell'applicazione in questione.

L'Ingegneria del Software è una branca dell'Ingegneria Informatica dedicata allo studio di metodi e tecniche per la produzione del software seguendo parametri di qualità ben definiti e standardizzati. A differenza di approcci più artigianali basati sull'esperienza e sull'iniziativa individuale del programmatore, l'Ingegneria del Software adotta processi sistematici e rigorosi per garantire la costruzione di software affidabile, efficiente e sicuro.

L'Ingegneria del Software mira a migliorare la produttività, ridurre gli errori e assicurare che il software sviluppato sia conforme alle specifiche richieste e agli standard di qualità stabiliti.

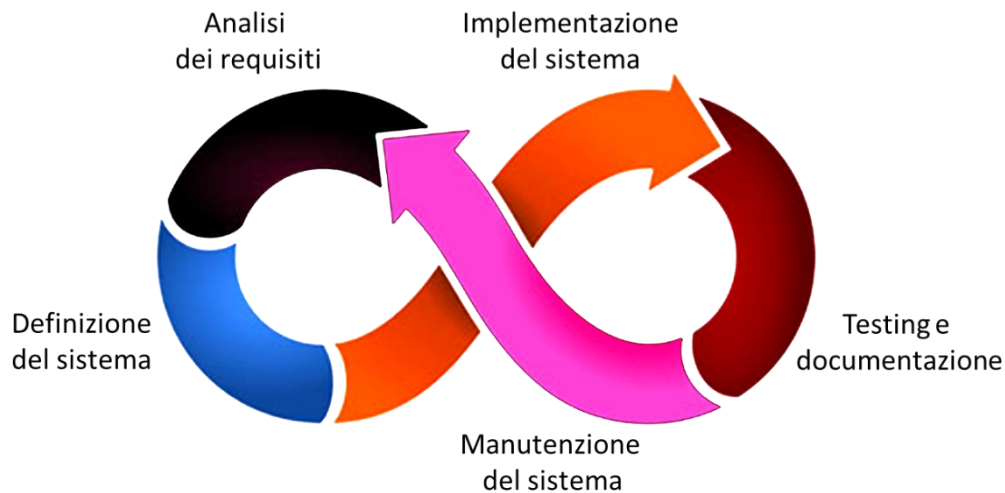


Figura 1 – Il ciclo di vita del software

Come riportato in Fig. 1 e in Fig.2, il ciclo di vita del software viene tipicamente suddiviso in diverse fasi o sotto-attività interdipendenti e coordinate tra loro, allo scopo di gestire efficacemente tutte le fasi dal concepimento alla distribuzione e alla manutenzione del prodotto software.

Le fasi principali del ciclo di vita del software sono:

- l'analisi dei requisiti;
- la progettazione;
- l'implementazione;
- il testing;
- la manutenzione.

Ognuna di queste fasi contribuisce al raggiungimento degli obiettivi definiti per il software, assicurando che il prodotto finale sia di alta qualità, efficiente e soddisfi le esigenze degli utenti.

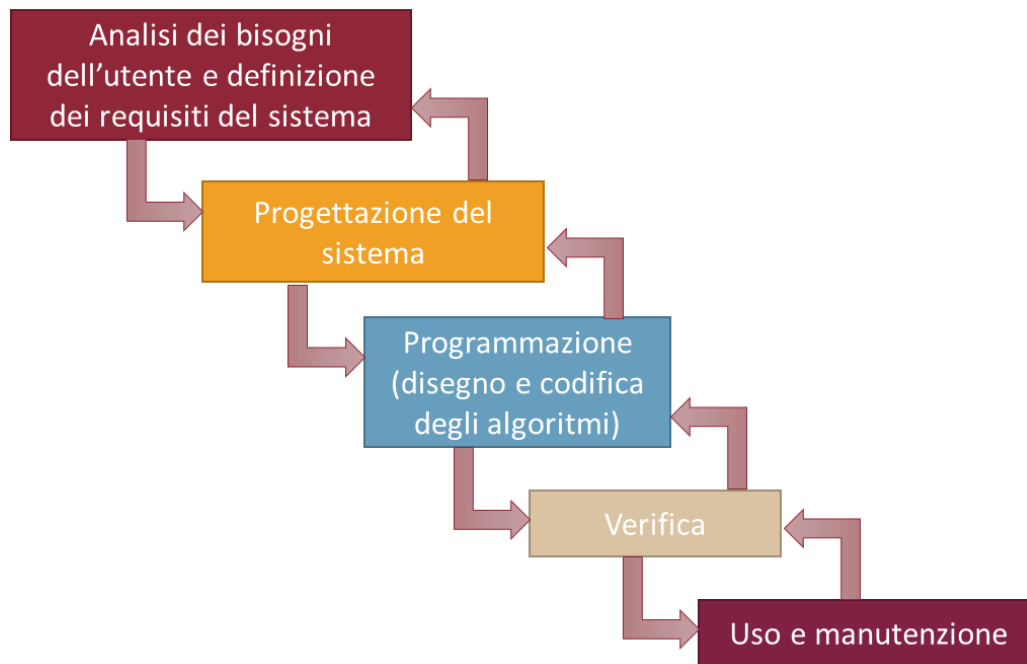


Figura 2 – Le fasi principali del ciclo di vita del software

L'analisi dei requisiti e delle funzionalità di un programma è un processo cruciale che identifica in maniera non ambigua cosa il software deve fare. Questo processo coinvolge l'acquisizione delle informazioni necessarie per comprendere a fondo il problema da risolvere. Tali informazioni vengono raccolte attraverso colloqui con utenti di diversi livelli, garantendo una visione completa delle loro esigenze e aspettative. Inoltre, un esame approfondito dell'ambiente in cui il programma sarà utilizzato fornisce ulteriori dettagli e contesto, assicurando che il software sia adeguato e funzionale nel suo specifico scenario operativo.

I requisiti funzionali descrivono cosa deve fare il programma e su quali dati deve operare, specificando le azioni che il sistema deve essere in grado di compiere.

D'altra parte, i requisiti non funzionali delineano le prestazioni che il programma deve offrire, come l'efficienza, la sicurezza e l'affidabilità. Entrambe le tipologie di requisiti sono essenziali per garantire che il software sviluppato soddisfi le aspettative e le necessità degli utenti finali.

Le specifiche ottenute nella fase di analisi devono essere formulate in modo da avere un'unica interpretazione evitando qualsiasi ambiguità o inconsistenza. È essenziale che queste specifiche coprano tutte le possibili situazioni coinvolte nel problema, assicurando una visione completa e accurata delle esigenze.

Per facilitare il confronto tra il programma e l'utente, è necessario semplificare gli effetti delle trasformazioni tra dati di ingresso e di uscita, rendendo chiaro e comprensibile come i dati vengono

elaborati e quali risultati finali ci si può aspettare. Questo approccio garantisce che le aspettative dell'utente siano allineate con le funzionalità del programma, promuovendo una comunicazione efficace e una maggiore soddisfazione del cliente.

Al termine della fase di analisi, è essenziale disporre di una documentazione completa che includa la definizione dettagliata dei dati di ingresso al problema e dei dati in uscita dal problema. Inoltre, questa documentazione deve contenere una descrizione chiara e precisa del metodo risolutivo proposto, che, almeno teoricamente, dovrebbe soddisfare le specifiche del problema.

La documentazione è cruciale per garantire che tutte le parti coinvolte abbiano una comprensione condivisa delle condizioni iniziali, degli obiettivi finali e del processo attraverso il quale si intende raggiungere la soluzione desiderata.

2. La progettazione del software

La progettazione del software è un'attività complessa che richiede una fase iniziale di attenta analisi del problema e un continuo processo di valutazione delle decisioni da prendere. In fase progettuale, è fondamentale mantenere una netta separazione tra analisi e sintesi, ovvero tra il "cosa" e il "come".

La fase del "cosa" riguarda l'analisi dei requisiti e la definizione delle specifiche funzionali, identificando ciò che il software deve fare. La fase del "come", invece, si concentra sul progetto a diversi livelli di dettaglio, determinando come il software realizzerà requisiti e specifiche. Questa distinzione garantisce una progettazione più strutturata e coerente, facilitando la realizzazione di un software efficace e affidabile.

Nella progettazione del software esistono due approcci principali.

L'approccio Top-Down con raffinamenti successivi, in cui il problema complessivo viene suddiviso in componenti più piccoli e gestibili. Ogni elemento scomposto diventa un nuovo problema da analizzare e risolvere, portando a una continua suddivisione in problemi più elementari. Questo processo iterativo prosegue fino a raggiungere un livello di scomposizione che sia comprensibile per l'esecutore, facilitando così la realizzazione di un prodotto software in uso.

La progettazione Top-Down segue un processo iterativo in cui l'algoritmo viene progressivamente decomposto in versioni sempre più dettagliate, avvicinandosi alla sua espressione finale nel linguaggio di programmazione. In pratica, nelle fasi iniziali si ha un'astrazione dei dettagli essenziali del sistema e nell'omissione di quelli non essenziali, creando una visione stratificata del sistema. Ogni livello della progettazione presenta una visione astratta dei livelli sottostanti: i moduli di livello più alto specificano gli obiettivi di determinate azioni o cosa deve essere fatto, mentre i moduli di livello più basso descrivono come tali obiettivi verranno raggiunti. Questo approccio permette di gestire la complessità del software in modo organizzato e sistematico, facilitando la comprensione e la realizzazione del progetto.

Ogni sottoproblema può essere descritto sinteticamente ed espressivamente in linguaggio naturale o come un insieme di istruzioni in linguaggio di programmazione, se la funzionalità è facilmente descrivibile. Nel primo caso, il sottoproblema diventa un nuovo problema da affrontare separatamente. Per garantire una corretta integrazione, è fondamentale definire e documentare le variabili di ingresso e di uscita, nonché le interfacce tra i vari sottoproblemi, assicurando che ogni componente del sistema possa comunicare e interagire efficacemente con gli altri.

Ad ogni passo di questa decomposizione, è essenziale organizzare i sottoproblemi in tre strutture principali:

- sequenziale, se l'analisi del problema indica che le attività devono essere svolte una dopo l'altra;
- alternativa, se è necessario scegliere tra una o più attività;
- iterativa, se una o più attività devono essere ripetute più volte.

La Fig. 3 riporta un esempio di applicazione dell'approccio Top-Down applicato al calcolo di una percentuale.

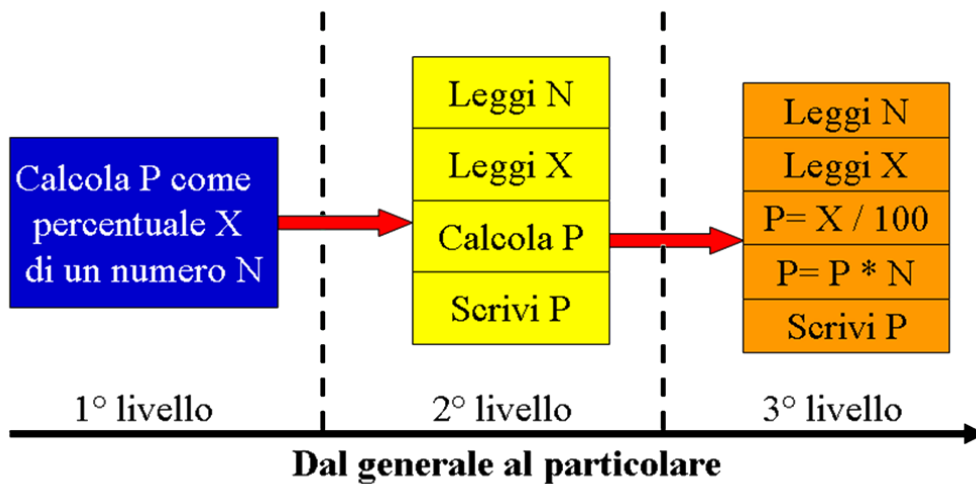


Figura 3 – Esempio di approccio top-down nel calcolo di una percentuale

Come riportato in Fig. 4, il processo di raffinamenti successivi può essere efficacemente rappresentato come un albero, dove la radice rappresenta il problema principale, i nodi corrispondono alle decisioni di progetto prese a ciascun livello di dettaglio, e le foglie rappresentano la descrizione finale della soluzione, formulata in modo comprensibile per l'esecutore.

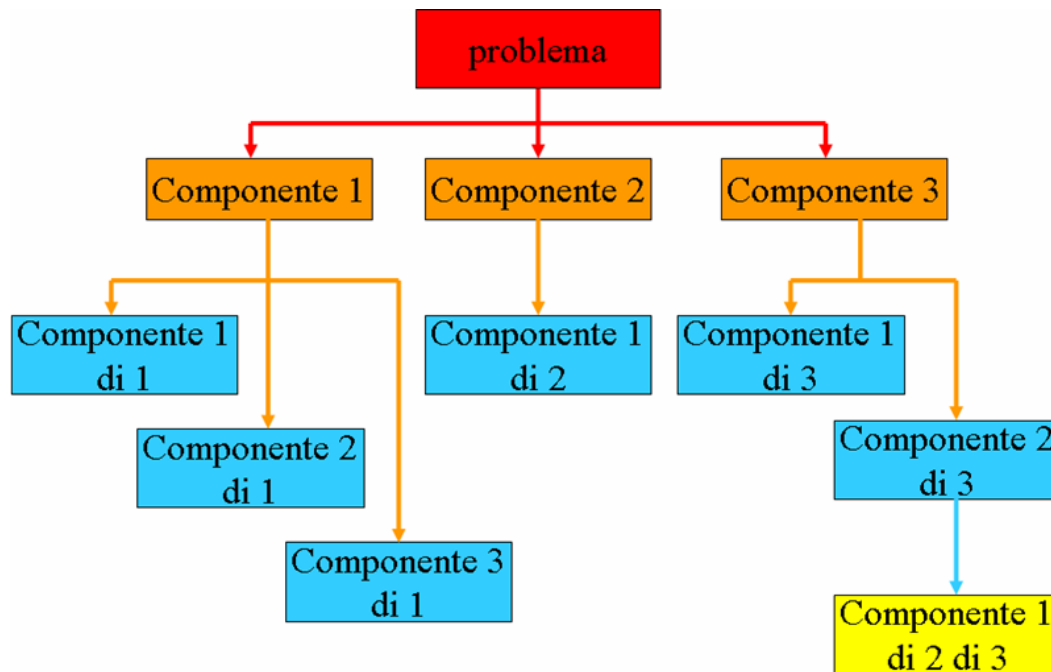


Figura 4 – Rappresentazione ad albero dei raffinamenti successivi nella progettazione Top-Down

La progettazione del software Bottom-Up adotta un approccio induttivo, iniziando dal sistema reale a disposizione e procedendo verso la soluzione completa. In questo metodo, si creano progressivamente moduli elementari, che vengono poi integrati per formare moduli capaci di compiere operazioni più complesse.

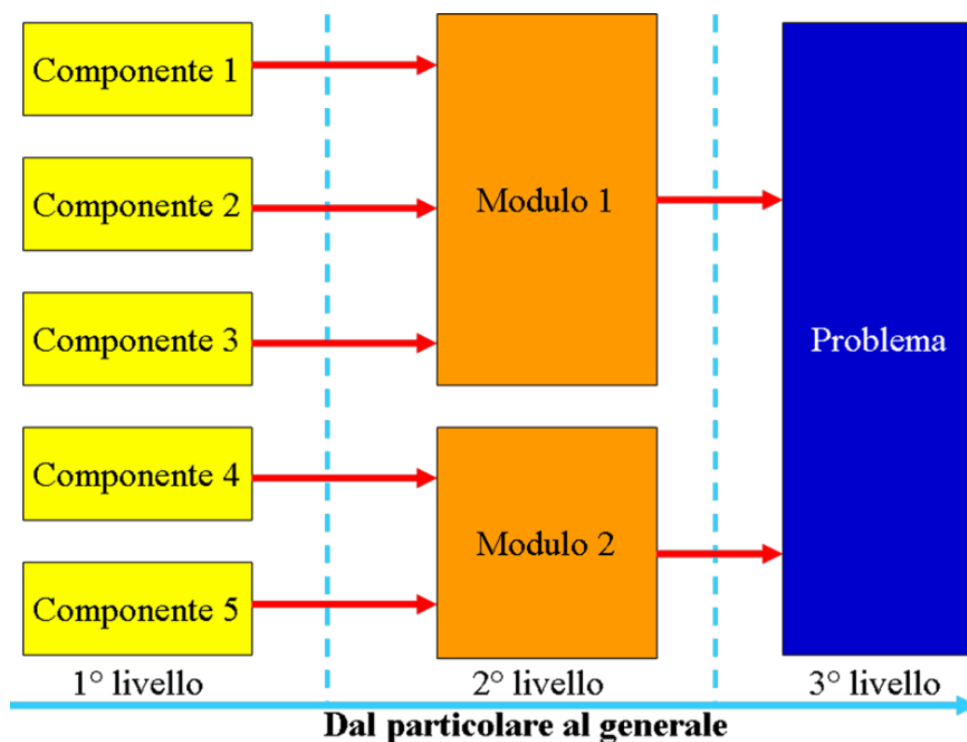


Figura 5 – Esempio di approccio top-down nel calcolo di una percentuale

Il processo continua iterativamente, sviluppando moduli sempre più avanzati, fino a quando si ottiene un modulo finale che corrisponde alla soluzione al problema.

Seguendo una rappresentazione ad albero (vedi Fig. 5, la progettazione Bottom-Up parte dalle foglie, ossia dai componenti più semplici, e procede verso la radice, integrando e raffinando i moduli fino a raggiungere la soluzione desiderata.

3. La programmazione strutturata

La programmazione strutturata è una metodologia per realizzare codice di alta qualità, promuovendo pratiche che migliorano la leggibilità, la documentabilità, la modificabilità e la provabilità del software.

Un codice leggibile è più facile da comprendere per i programmatori, facilitando la manutenzione e l'aggiornamento. La documentabilità garantisce che il codice sia accompagnato da spiegazioni chiare, rendendo più semplice il passaggio di conoscenze tra i membri del team. La modificabilità assicura che il software possa essere facilmente adattato a nuove esigenze o miglioramenti senza compromettere la sua struttura originale. Infine, la provabilità implica che il codice sia progettato in modo da poter essere testato e verificato con facilità, assicurando così la sua affidabilità e correttezza.

La programmazione strutturata si basa sull'uso di strutture di controllo con un unico punto di ingresso e di uscita, garantendo un flusso logico e prevedibile all'interno del programma. La programmazione strutturata incentiva inoltre l'approccio alla progettazione top-down o bottom-up, consentendo agli sviluppatori di suddividere i problemi complessi in sottoproblemi più gestibili e organizzare il codice in moduli ben definiti.

I principi della programmazione strutturata non solo migliorano la leggibilità e la robustezza del codice, ma facilitano anche il processo di sviluppo, debugging e aggiornamento del software.

La modularità del codice è un principio fondamentale della programmazione strutturata che prevede l'organizzazione del codice in moduli funzionali, ciascuno dei quali ha un compito singolo e ben definito. Ogni modulo deve essere dotato di un solo ingresso e una sola uscita, il che facilita la creazione di una struttura regolare e coerente nei programmi. Questo approccio non solo supporta efficacemente la fase di progettazione, ma riflette anche la limitazione umana di concentrarsi su un solo aspetto di un problema alla volta, rendendo il codice più comprensibile e gestibile. La modularità, quindi, non solo migliora la leggibilità e la manutenibilità del codice, ma consente anche agli sviluppatori di isolare e risolvere problemi specifici senza interferire con altri componenti del sistema.

Nella programmazione strutturata le strutture di controllo devono assolutamente avere un solo punto di ingresso e un solo punto di uscita, assicurando un flusso logico e prevedibile del controllo del programma. Inoltre, l'integrazione dei vari moduli tramite le strutture di controllo deve produrre un modulo risultante che mantenga anch'esso la caratteristica di avere un unico punto di ingresso e un unico punto di uscita. Questo principio è cruciale per la corretta esecuzione e manutenzione del software, facilitando la leggibilità e la modularità del codice.

La documentazione è uno strumento fondamentale per garantire la chiarezza e la leggibilità del codice, permettendo una maggiore comprensione del problema che il programma risolve e, di conseguenza, della sua correttezza. Una buona documentazione facilita la ripresa del progetto dopo eventuali interruzioni, supporta una comunicazione più semplice delle scelte di progetto e consente di modificare il programma più agevolmente in risposta a variazioni nelle specifiche del problema. In questo modo, la documentazione non solo migliora la qualità e la manutenzione del software, ma assicura anche che il lavoro svolto possa essere compreso e continuato efficacemente da altri sviluppatori.

La documentazione di un progetto deve essere prodotta durante tutte le fasi di sviluppo per garantire la sua efficacia e completezza, assicurando che le scelte vengano documentate nel momento stesso in cui vengono fatte. Per una gestione e un aggiornamento ottimali del codice, è essenziale integrare la documentazione direttamente all'interno del codice stesso. Questa pratica consente di mantenere il codice facilmente comprensibile e manutenibile.

La documentazione deve essere articolata su due livelli: una documentazione esterna del programma, che fornisce una panoramica generale e dettagli di alto livello, e una documentazione interna del programma, che descrive in dettaglio il funzionamento e l'implementazione del codice.

La documentazione esterna è il primo livello di documentazione e va compilata preliminarmente nella fase di analisi dei requisiti. Essa descrive soltanto cosa fa il programma e non come lo fa, fornendo dettagli operativi che rendono gli utenti autonomi nell'utilizzo del programma.

La documentazione esterna deve includere informazioni sulle funzionalità esterne, le modalità di attivazione del programma, le diagnostiche di errore, la configurazione richiesta del sistema, le indicazioni sull'installazione del programma, nonché la versione e la data di aggiornamento. Questi elementi sono essenziali per garantire che gli utenti possano utilizzare efficacemente il software senza necessitare di ulteriori supporti tecnici.

La documentazione interna di un programma riveste un ruolo cruciale nel fornire una comprensione dettagliata della sua struttura e funzionamento. Essa descrive la struttura interna del programma attraverso le scelte relative alle strutture dati e agli algoritmi adottati, offrendo una mappa chiara delle decisioni prese durante lo sviluppo. L'indentazione del codice svolge un ruolo essenziale nel rendere visibile la struttura del codice stesso, facilitando la comprensione della sua organizzazione e logica. La documentazione segue un approccio top-down, delineando come il programma è stato generato attraverso i vari passaggi di raffinamento e implementazione. L'uso di nomi di variabili autoesplicativi contribuisce a rendere il codice più comprensibile, mentre i commenti integrati forniscono spiegazioni dettagliate e contestuali delle porzioni di codice.

Le motivazioni dietro le scelte implementative sono chiaramente documentate, fornendo un contesto per comprendere il "perché" di determinate decisioni. Infine, l'uso di asserzioni aiuta a garantire la correttezza del programma, fornendo punti di controllo che verificano le condizioni attese durante l'esecuzione. In sintesi, la documentazione interna non solo rende il programma più accessibile e manutenibile, ma ne assicura anche la comprensione completa e la corretta esecuzione nel tempo.

Bibliografia

- Le radici dell'informatica. Dal bit alla programmazione strutturata. Angelo Chianese, Vincenzo Moscato, Antonio Picariello, Carlo Sansone. Maggioli Editore.